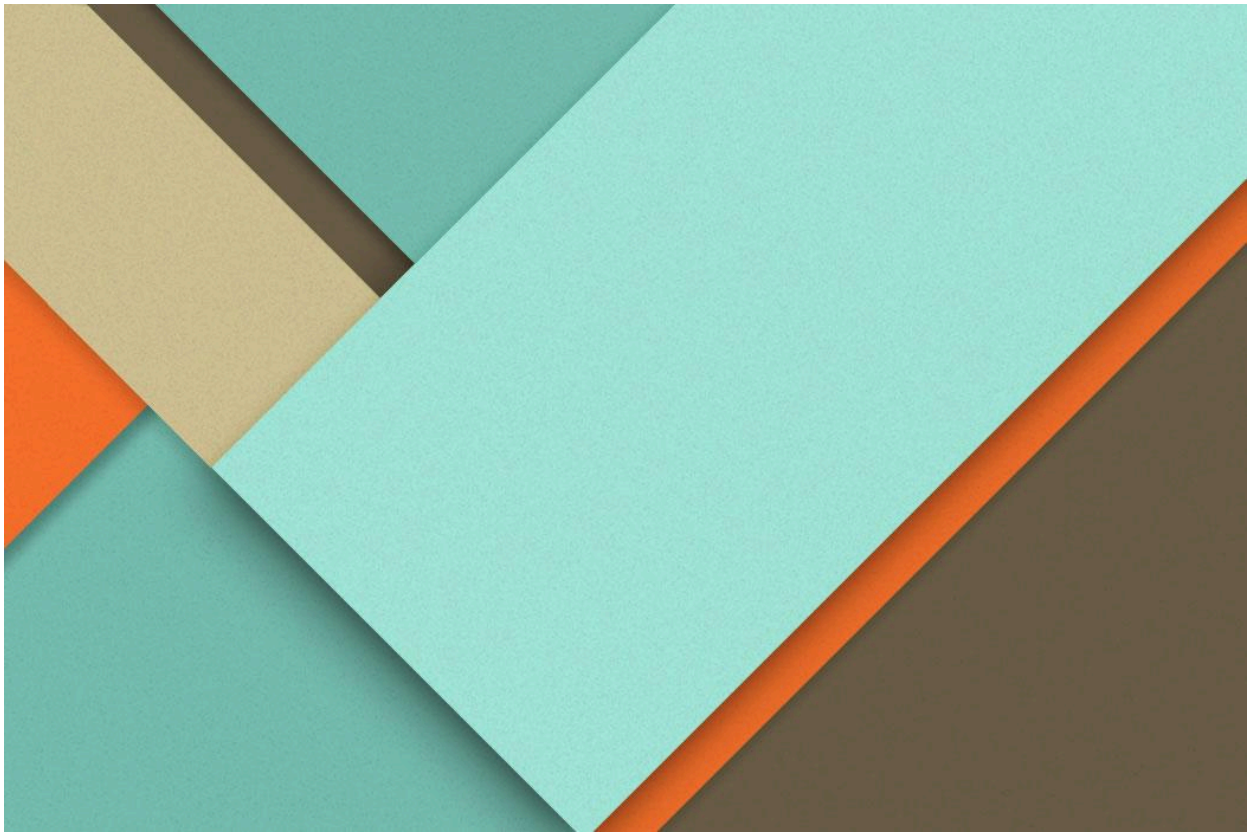


GoScript IDE – Base del Proyecto

25/04/2026

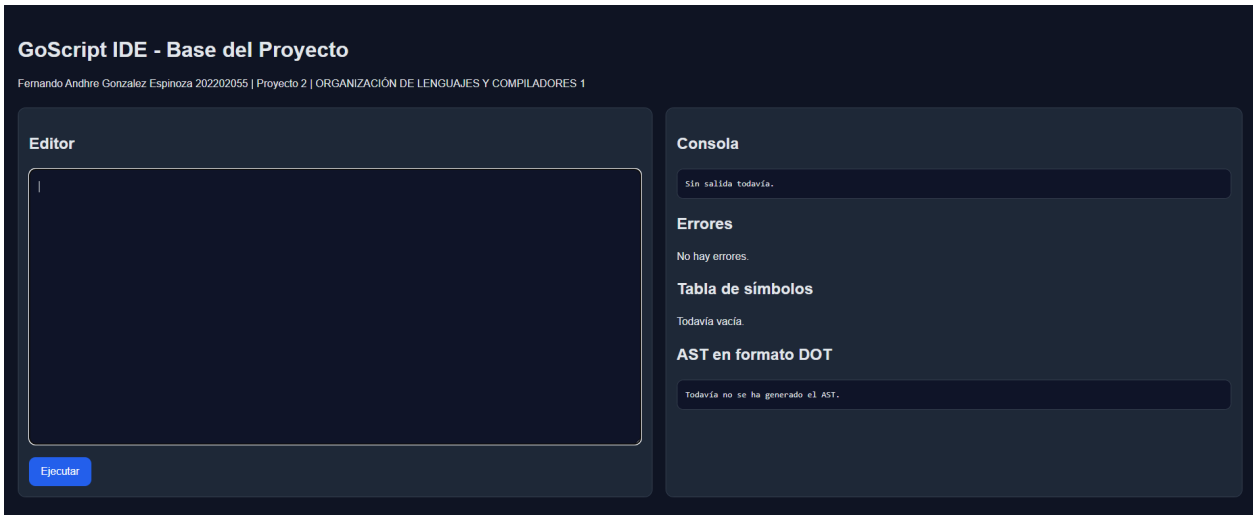
Fernando Andhre
Gonzalez Espinoza
202202055
ORGANIZACIÓN DE LENGUAJES Y COMPILADORES 1

1. Introducción

El presente Manual de Usuario tiene como finalidad explicar el funcionamiento del sistema GoScript IDE – Base del Proyecto, una aplicación diseñada para permitir la escritura, análisis y ejecución controlada de código fuente en un lenguaje tipo GoScript.

La herramienta proporciona un entorno visual en el que el usuario puede ingresar instrucciones, ejecutar el análisis del programa y visualizar diferentes resultados, tales como la salida en consola, la tabla de errores detectados, la tabla de símbolos generada y el Árbol de Sintaxis Abstracta en formato DOT.

Este manual está orientado al usuario final, por lo que describe de forma clara y ordenada el uso del sistema, su interfaz, sus funciones principales y la interpretación de los resultados que se muestran en pantalla.



2. Objetivos del sistema

2.1 Objetivo general

- Proporcionar un entorno gráfico que permita escribir, analizar y ejecutar programas en GoScript, facilitando la visualización de resultados y la detección de errores léxicos, sintácticos y semánticos.

2.2 Objetivos específicos

- Permitir el ingreso de código fuente mediante un editor integrado.
- Ejecutar el análisis del código ingresado.
- Mostrar la salida producida por el programa en una consola visual.
- Reportar errores encontrados durante el análisis.
- Presentar la tabla de símbolos generada a partir del programa.
- Mostrar el AST del programa en formato DOT para fines de análisis y depuración.

3. Requisitos para el uso del sistema

Para utilizar correctamente el sistema, se recomienda contar con lo siguiente:

- Un equipo de cómputo con sistema operativo Windows, Linux o macOS.
- Un navegador web actualizado, como Google Chrome, Microsoft Edge o Mozilla Firefox.
- El frontend y el backend del proyecto en ejecución local.
- Conexión correcta entre la interfaz y el servidor del sistema.

En caso de ejecutar el proyecto localmente, también se requiere:

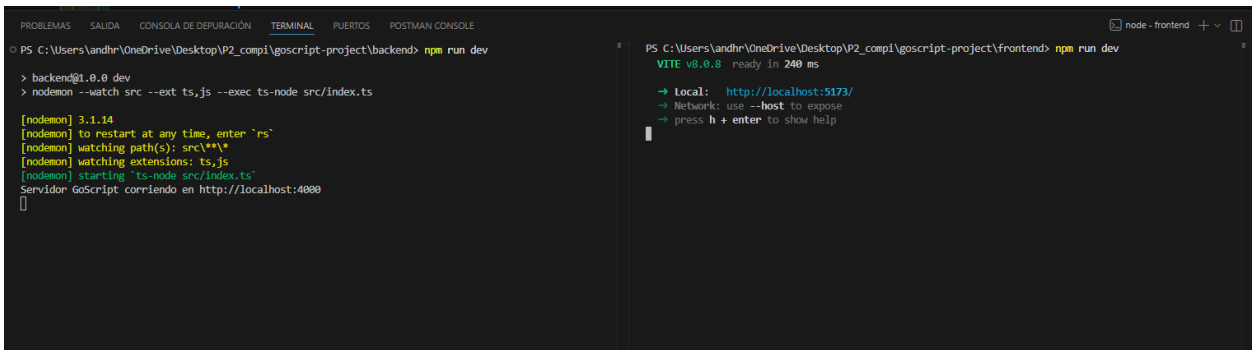
- Node.js instalado.
- Dependencias del proyecto instaladas.
- Parser generado correctamente.

4. Acceso al sistema

Para acceder al sistema, el usuario debe seguir estos pasos:

- Abrir una terminal en la carpeta del backend del proyecto.
- Ejecutar el comando correspondiente para iniciar el servidor.
- Abrir una segunda terminal, si aplica, para iniciar el frontend.
- Ingresar desde el navegador a la dirección local en la que se encuentre corriendo la aplicación.

- Una vez cargada la página, se mostrará la interfaz principal del sistema, compuesta por un editor de texto y varias áreas de salida.



5. Descripción general de la interfaz

La interfaz del sistema está dividida en varias secciones, cada una con una función específica.

5.1 Editor

Es el área principal donde el usuario escribe o pega el código fuente que desea analizar. En esta sección se introducen las instrucciones del programa en GoScript.

5.2 Botón “Ejecutar”

Este botón permite enviar el código fuente al backend para que sea procesado. Una vez presionado, el sistema realiza el análisis y actualiza las demás secciones con los resultados obtenidos.

5.3 Consola

Muestra la salida generada por el programa. Aquí aparecen los valores impresos con instrucciones como fmt.Println(...) y otros resultados producidos durante la ejecución.

5.4 Errores

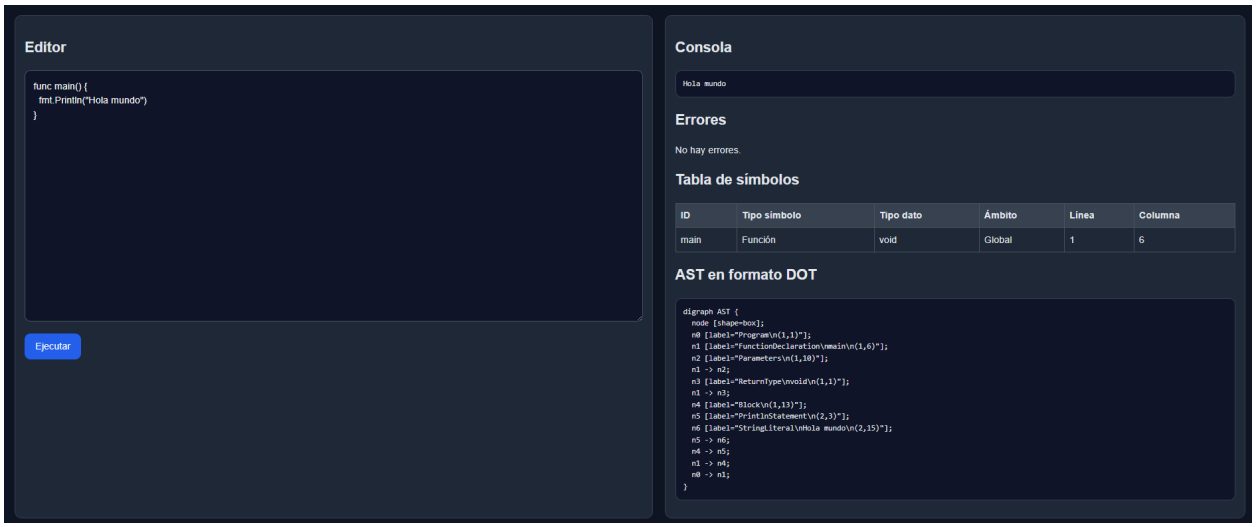
En esta sección se presenta una tabla con los errores encontrados. Cada error incluye información como el tipo de error, su descripción, la línea y la columna donde ocurrió.

5.5 Tabla de símbolos

Muestra las variables, funciones, parámetros, structs y otros identificadores reconocidos por el sistema, junto con su tipo de dato, ámbito, línea y columna.

5.6 AST en formato DOT

Presenta la representación textual del Árbol de Sintaxis Abstracta generado a partir del código ingresado. Esta salida es útil principalmente para revisión estructural y análisis del programa.



6. Funcionalidades principales

El sistema cuenta con varias funciones que facilitan el análisis del código fuente.

6.1 Ingreso de código

El usuario puede escribir manualmente el código en el editor o pegarlo desde otro archivo o documento.

6.2 Ejecución del análisis

Al presionar el botón de ejecución, el sistema procesa el código y determina si es correcto o si contiene errores.

6.3 Visualización de salida

Si el programa es válido y contiene instrucciones de impresión, la salida se mostrará en la consola.

6.4 Detección de errores

Si existen errores en el programa, estos aparecerán en la tabla correspondiente con la ubicación exacta del problema.

6.5 Visualización de símbolos

El sistema registra información relevante de los identificadores utilizados y la presenta en forma de tabla.

6.6 Visualización estructural del programa

Además de la salida textual, el sistema genera una representación del AST en formato DOT.

7. Procedimiento de uso

A continuación, se describe el procedimiento general para utilizar el sistema.

Paso 1. Ingresar el código

En el editor, escribir o pegar el programa que se desea analizar.

Ejemplo:

```
func main() {
a := 10
b := 20
fmt.Println(a + b)
}
```

Paso 2. Ejecutar el análisis

Presionar el botón Ejecutar.

Paso 3. Revisar la consola

Si el código es correcto, la consola mostrará el resultado esperado.

En el ejemplo anterior, la salida será:

```
30
```

Paso 4. Revisar errores

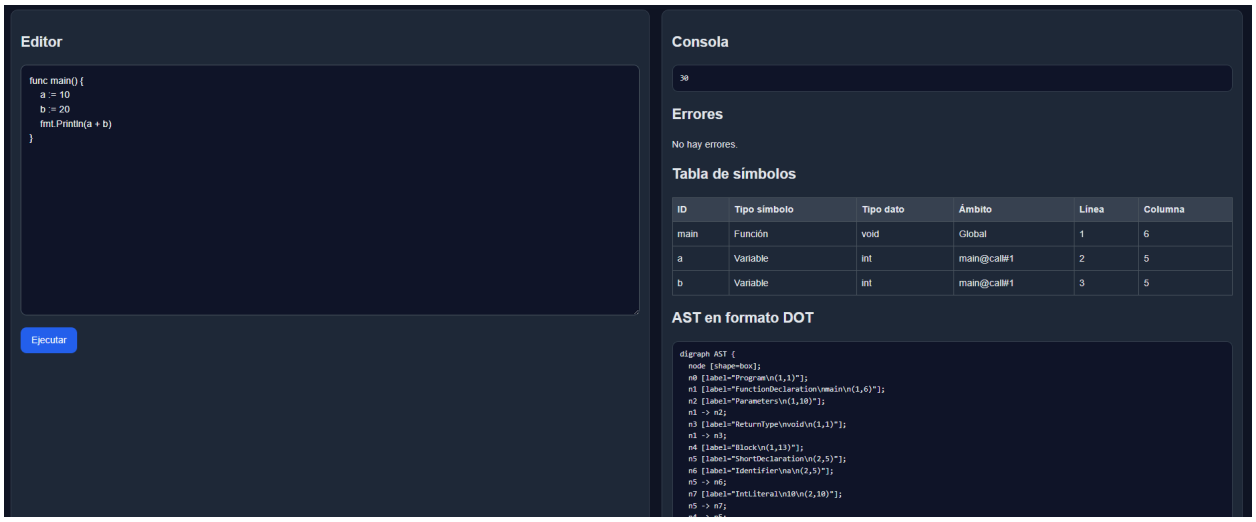
Si el programa contiene errores, estos se mostrarán en la tabla de errores.

Paso 5. Revisar tabla de símbolos

Verificar que las variables, funciones y demás elementos del programa hayan sido reconocidos correctamente.

Paso 6. Revisar AST

Observar la estructura sintáctica del programa en formato DOT.



8. Ejemplos de uso

8.1 Ejemplo de programa correcto

```
func main() {
nombre := "Luis"
edad := 21
fmt.Println(nombre)
fmt.Println(edad)
}
```

Resultado esperado en consola

```
Luis
21
```

8.2 Ejemplo con error semántico

```
func main() {
x := 10
x = "hola"
fmt.Println(x)
```

```
}
```

Resultado esperado

El sistema debe reportar un error indicando que no se puede asignar un valor de tipo string a una variable de tipo int.

8.3 Ejemplo con estructura de control

```
func main() {
contador := 0

for contador < 5 {
fmt.Println(contador)
contador++
}
}
```

Resultado esperado en consola

```
0
1
2
3
4
```

8.4 Ejemplo con structs

```
struct Persona {
string nombre;
int edad;
}

func main() {
```

Persona p = { nombre: "Ana", edad: 20 }

fmt.Println(p.nombre)

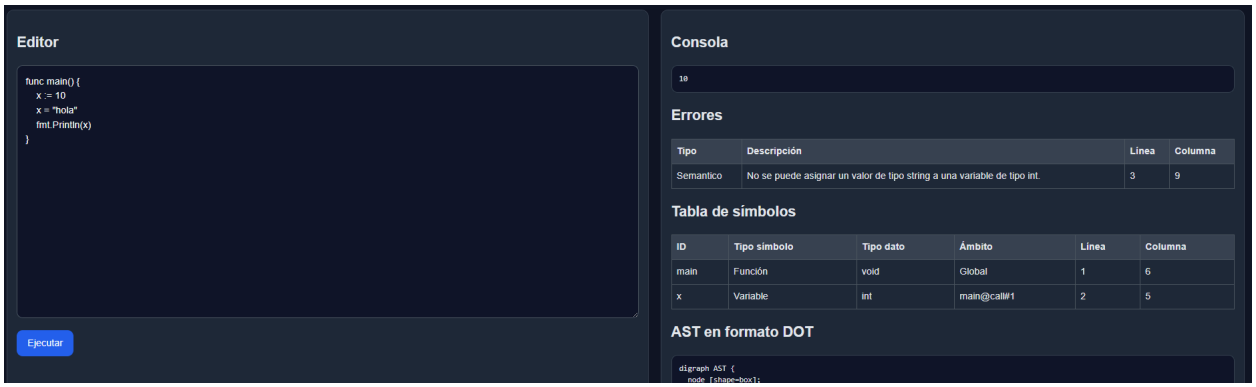
fmt.Println(p.edad)

}

Resultado esperado en consola

Ana

20



9. Interpretación de resultados

9.1 Consola

La consola muestra los resultados del programa. Si no hay instrucciones de salida o si ocurre un error antes de ejecutar la impresión, es posible que la consola no muestra contenido útil.

9.2 Tabla de errores

Cada fila representa un error detectado. Los campos más comunes son:

- Tipo: indica si el error es léxico, sintáctico o semántico.
- Descripción: explica el problema encontrado.
- Línea: muestra en qué línea ocurrió el error.

- Columna: muestra en qué posición aproximada ocurrió el error.

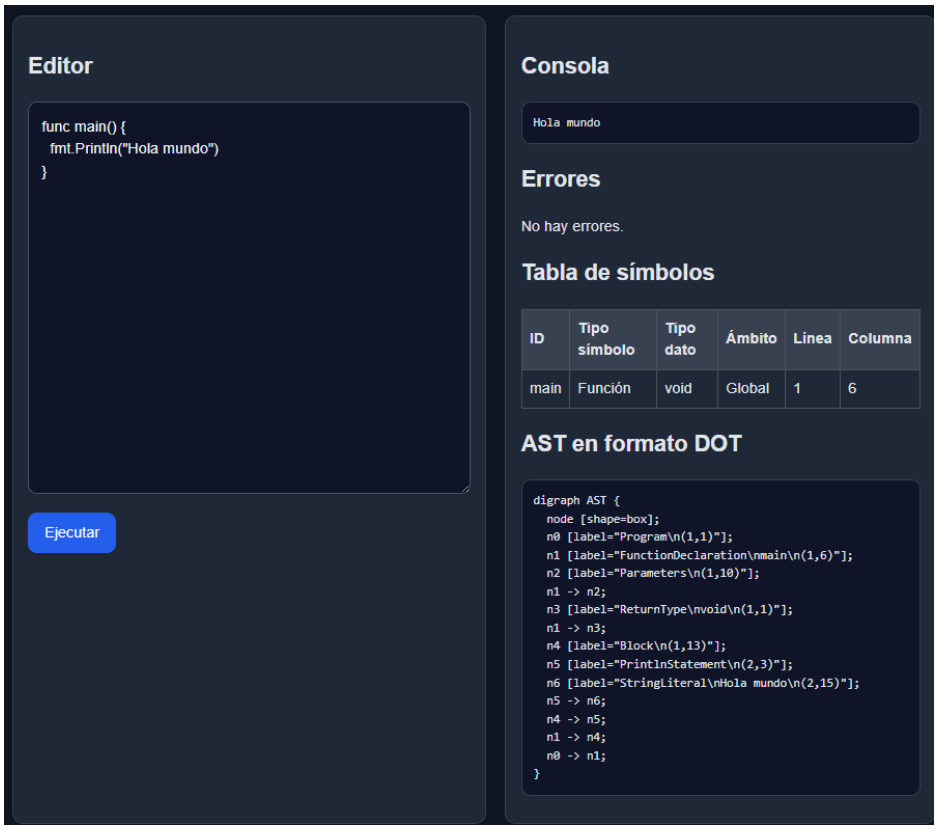
9.3 Tabla de símbolos

La tabla de símbolos ayuda a verificar qué elementos fueron reconocidos correctamente por el sistema. Normalmente incluye:

- ID: nombre del identificador.
- Tipo símbolo: variable, función, parámetro, struct, entre otros.
- Tipo dato: tipo asociado, como int, string, float64, etc.
- Ámbito: lugar donde existe el identificador.
- Línea y columna: ubicación de su declaración o registro.

9.4 AST en formato DOT

El AST en formato DOT representa la estructura interna del programa. Aunque está pensado principalmente para fines de revisión técnica, puede ser útil para verificar si el programa fue interpretado estructuralmente de forma correcta.



10. Manejo de errores comunes

A continuación, se describen algunos errores frecuentes y su posible solución.

10.1 Variable no declarada

Ocurre cuando se utiliza una variable que no ha sido definida previamente.

Ejemplo

```
func main() {
 fmt.Println(x)
}
```

Solución

Declarar la variable antes de usarla.

10.2 Incompatibilidad de tipos

Ocurre cuando se intenta asignar o combinar valores de tipos incompatibles.

Ejemplo

```
func main() {
x := 10
x = "texto"
}
```

Solución

Usar valores del mismo tipo o realizar la conversión correspondiente si el sistema lo permite.

10.3 Índice fuera de rango

Ocurre al intentar acceder a una posición inexistente dentro de un arreglo o slice.

Ejemplo

```
func main() {
nums := []int{1, 2, 3}
fmt.Println(nums[10])
}
```

Solución

Verificar que el índice se encuentre dentro de los límites válidos.

10.4 Error de sintaxis

Se produce cuando la estructura del código no cumple con la gramática esperada.

Ejemplo

```
func main( {
fmt.Println("Hola")
}
```

Solución

Revisar signos, paréntesis, llaves, comas y punto y coma según la sintaxis del lenguaje.

10.5 Uso incorrecto de range

Ocurre cuando range se aplica sobre un valor que no es iterable.

Ejemplo

```
func main() {
x := 100
for i, v := range x {
fmt.Println(i, v)
}
}
```

Solución

Usar range únicamente sobre arreglos, slices o estructuras soportadas por el sistema.

11. Recomendaciones de uso

- Escribir el código de manera ordenada y clara.
- Probar primero programas pequeños antes de cargar pruebas más grandes.
- Revisar la tabla de errores antes de interpretar la salida de la consola.
- Utilizar nombres de variables descriptivos para facilitar la lectura.
- Verificar que el backend esté corriendo correctamente antes de ejecutar pruebas.
- Guardar versiones del código a medida que se vayan agregando nuevas funciones.

12. Glosario básico

- Analizador léxico: componente que reconoce tokens o unidades básicas del lenguaje.
- Analizador sintáctico: componente que verifica si el orden del código cumple la gramática definida.
- Error semántico: error relacionado con el significado del programa, por ejemplo, tipos incompatibles o uso incorrecto de variables.
- Tabla de símbolos: estructura que almacena información sobre variables, funciones y otros identificadores.

- AST: Árbol de Sintaxis Abstracta que representa la estructura del programa.
- DOT: formato textual utilizado para representar grafos o árboles.
- Slice: estructura dinámica similar a un arreglo.
- Struct: estructura que agrupa varios campos bajo un mismo tipo.

13. Conclusiones

El sistema GoScript IDE – Base del Proyecto proporciona una interfaz amigable para escribir, analizar y ejecutar código GoScript, ofreciendo al usuario herramientas visuales útiles para comprender el comportamiento del programa.

Entre sus principales ventajas se encuentran la facilidad para detectar errores, la visualización de la salida del programa, la generación de la tabla de símbolos y la representación del AST. Estas características convierten al sistema en una herramienta adecuada para el aprendizaje, validación y depuración de programas dentro del entorno del proyecto académico.

El uso correcto del sistema, acompañado de una lectura adecuada de los errores y resultados, permite al usuario desarrollar programas de forma más ordenada y eficiente.